

Nirikshak: An Industrial IoT & Predictive Maintenance Framework

Samarth Maheshwari¹, Anisha Pansare¹, Shriraj Dongare¹ and Rupali Kawade²

¹*Dept. of Electronics and Telecommunication Engineering, Pimpri Chinchwad College Of Engineering Research, Ravet, Pune, Maharashtra, India*

²*Assistant Professor, Dept. of Electronics and Telecommunication Engineering, Pimpri Chinchwad College of Engineering Research (PCCOER), Ravet, Pune, Maharashtra, India
{samarthmaheshwari58, anishapansare1512, shrirajdongare2004, rupali2118}@gmail.com*

Keywords: Industrial IoT, Predictive Maintenance, Edge Computing, Machine Learning, MQTT Communication, Anomaly Detection, Fault Prediction, Remaining Useful Life (RUL), Smart Manufacturing, Real-Time Monitoring

Abstract: Predictive maintenance (PdM) has emerged as a critical approach for reducing unplanned downtime and improving operational efficiency in industrial systems. However, the adoption of PdM remains limited, particularly among small and medium-sized enterprises (SMEs), due to the high cost, proprietary nature, and implementation complexity of existing commercial solutions. These systems often require significant capital investment and specialized expertise, creating a substantial barrier to entry. This paper presents a low-cost, scalable Industrial IoT-based predictive maintenance system that integrates multi-modal sensors, MQTT-based communication, and machine learning techniques for real-time condition monitoring and fault prediction. The proposed system utilizes an embedded edge device for data acquisition and preprocessing, while cloud-based analytics enable anomaly detection and fault classification. Experimental evaluation demonstrates a fault detection accuracy of 94.2%, with an end-to-end system latency of 320 ms. The total system cost is approximately Rs.8,500, representing a significant reduction compared to conventional PdM solutions. The proposed approach offers an affordable and practical alternative for SMEs, enabling early fault detection, reduced maintenance costs, and improved equipment reliability. This work contributes toward democratizing predictive maintenance through accessible, open, and efficient technological solutions.

1 INTRODUCTION

Modern industrial operations rely heavily on continuous and reliable machine operation to maintain productivity and reduce operational costs. Predictive maintenance (PdM) has emerged as an effective strategy to reduce equipment failures and optimize maintenance schedules (Theissler et al., 2021), (Samatas et al., 2021). However, despite its advantages, PdM adoption remains limited, particularly among small and medium-sized enterprises (SMEs), due to the high cost, proprietary nature, and technical complexity of existing commercial solutions.

Traditional maintenance strategies are broadly categorized into reactive and preventive approaches. Reactive maintenance, also known as run-to-failure, allows equipment to operate until breakdown, leading to unplanned downtime, high repair costs, and potential safety risks. Preventive maintenance, on the other hand, relies on fixed maintenance schedules regard-

less of the actual condition of the equipment, often resulting in unnecessary servicing, increased operational expenses, and inefficient utilization of component life.

Although predictive maintenance addresses these limitations by enabling condition-based monitoring and early fault detection (Aissani et al., 2024), (Benhanifia et al., 2025), current implementations are often expensive and lack accessibility. Moreover, many existing research efforts focus primarily on isolated machine learning models without providing a complete, real-time, and deployable system architecture.

To address these challenges, this paper proposes a low-cost, end-to-end IoT-based predictive maintenance system that integrates embedded edge devices (STM32), multi-modal sensors, MQTT-based communication, and machine learning algorithms (Elkateb et al., 2024), (Hamasha et al., 2025). The system is designed to provide real-time monitoring, anomaly detection, and fault prediction while maintaining afford-

ability and ease of deployment, making it suitable for SMEs and practical industrial applications.

2 LITERATURE REVIEW

2.1 Industrial IoT Architectures

Industrial IoT (IIoT)-based predictive maintenance systems are typically built using three architectural paradigms: cloud-based, edge-based, and hybrid architectures(Andriulo et al., 2024),(Savaglio et al., 2024). Cloud-centric systems rely on centralized processing, offering high computational power for complex analytics but suffering from high latency and bandwidth dependency. In contrast, edge-based architectures perform data processing near the data source, enabling low-latency responses and reduced network load. However, they are constrained by limited computational resources.

To overcome these limitations, hybrid edge–cloud architectures have emerged as the preferred approach(Andriulo et al., 2024),(Hamasha et al., 2025), combining real-time processing at the edge with scalable analytics in the cloud. This distributed model improves responsiveness while maintaining computational efficiency and scalability . Despite these advancements, implementing such architectures in a cost-effective and easily deployable manner remains a challenge.

2.2 Machine Learning Models for Predictive Maintenance

Machine learning (ML) plays a central role in predictive maintenance by analyzing sensor data to detect anomalies and predict failures(Theissler et al., 2021),(Taşcı et al., 2023),(Elkateb et al., 2024). Common supervised learning models include Random Forest, Support Vector Machines (SVM), and deep learning models such as Long Short-Term Memory (LSTM) networks(Taşcı et al., 2023),(Rakholia et al., 2025).

Random Forest models are widely used for fault classification due to their robustness and ability to handle nonlinear data. SVMs are effective for binary classification problems, especially with limited datasets. LSTM networks, on the other hand, are particularly suitable for time-series data(Taşcı et al., 2023), capturing temporal dependencies in sensor signals and achieving high prediction accuracy. However, deep learning models often require large datasets and high computational resources, making them less suitable for real-time embedded applications.

2.3 Commercial Systems vs Research Prototypes

Commercial predictive maintenance solutions, such as those offered by major industrial vendors, provide complete end-to-end platforms with integrated hardware, analytics, and visualization tools(Aissani et al., 2024),(Benhanifia et al., 2025). These systems typically achieve high accuracy and reliability but are associated with significant costs, often making them inaccessible to SMEs. Additionally, they are usually proprietary, limiting customization and scalability.

In contrast, academic and research-based systems primarily focus on developing and evaluating ML models or specific components of PdM systems(Theissler et al., 2021),(Benhanifia et al., 2025). While these approaches demonstrate promising accuracy and innovative techniques, they often lack full system integration, real-world validation, and deployability. Many research works emphasize algorithmic performance without addressing communication, real-time constraints, or user interface design.

2.4 Limitations of Existing Work

Despite significant progress, existing predictive maintenance solutions exhibit several limitations:

- **High Cost:** Commercial systems require substantial investment, restricting adoption in SMEs(Aissani et al., 2024),(Benhanifia et al., 2025).
- **Lack of Integration:** Research prototypes often focus on isolated components rather than complete end-to-end systems.
- **Limited Real-Time Capability:** Cloud-dependent architectures introduce latency, limiting real-time fault detection.
- **Complex Implementation:** Integration of IoT, ML, and communication systems requires significant expertise and infrastructure(Samatas et al., 2021),(Hamasha et al., 2025).

2.5 Research Gap

From the literature, it is evident that there is a lack of affordable, fully integrated, and real-time predictive maintenance systems suitable for practical deployment in SMEs(Benhanifia et al., 2025),(Hamasha et al., 2025). Existing solutions either provide high performance at high cost (commercial systems) or low-cost but incomplete implementations (research prototypes).

Therefore, there is a clear need for a cost-effective, end-to-end IoT-based predictive maintenance system that integrates sensing, edge processing, communication, and machine learning while ensuring real-time performance and ease of deployment.

3 Methodology

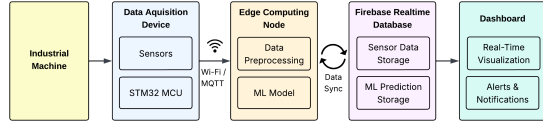


Figure 1: System Architecture

The suggested Nirikshak scheme involves the use of an Industrial IoT-driven predictive maintenance platform that includes sensor-based data collection, edge computing, MQTT protocol for data communication, machine learning algorithms for data analytics, cloud computing for storing data, and visualization of results via dashboards. The platform enables continuous monitoring of industrial machines and real-time fault detection based on a well-designed end-to-end solution pipeline.

3.1 Industrial Machine

The architecture begins with industrial machines like AC motors, compressors, pumps, and rotary machines. During their functioning, the machines create some physical and electrical variables such as vibration, temperature, rotation, change in voltage, and change in current among others. The variables serve as the indices of the state of the machines.

Monitoring the behavior of the machines continuously enables the recognition of unusual machine operation, which is related to factors such as bearing wearing, overheating, misalignment of the shaft, overload, and wear among others. Since most industrial problems occur gradually, it becomes easier to detect the faults at an early stage.

3.2 Data Acquisition Device

The process of collecting data uses a few sensors connected to an STM32 embedded controller system. A multiplicity of sensors is used in order to address the fact that faulty situations have an effect on different variables.

The analog sensor signals are processed using amplifiers and low-pass filters before being digitized us-

Table 1: Sensor Components Used

Sensor Type	Model	Description
Temperature	DS18B20	Temperature sensing
Vibration	MPU6050	Vibration monitoring
RPM	Proximity Sensor	Speed measurement
Voltage	ZMPT101B	Voltage monitoring
Current	SCT-013-030	Current monitoring

ing an ADC. Signal processing reduces external interference and improves the accuracy of the ADC. The microcontroller continuously captures the sensor signal values at specified times and holds the captured values in buffers.

Table 2: Hardware Specifications of Data Acquisition Unit

Component	Specification	Purpose
MCU	STM32F103	DAQ
Core	Cortex-M3	Processing
Clock	72 MHz	Sampling
ADC	12-bit	Conversion
RAM	20 KB	Buffering
Flash	64 KB	Storage
WiFi	ESP32	Communication
Supply	5V DC	Operation

Preprocessing such as noise filtering, normalization, and sensor calibration is done by the STM32 microcontroller before transferring the data. Preprocessing helps reduce communication overhead.

3.3 MQTT-Based Communication

The processed sensor data is transmitted wirelessly using the MQTT communication protocol. MQTT follows a lightweight publish-subscribe architecture and is widely adopted in Industrial IoT systems due to its low bandwidth utilization, low power consumption, and efficient real-time communication capability.

In the proposed implementation, the STM32-ESP32-based embedded device acts as the MQTT publisher and continuously publishes sensor readings in JSON format to the MQTT broker. The Mosquitto MQTT broker manages message distribution between connected devices. A Python-based edge computing application acts as the MQTT subscriber and continuously receives live sensor data for preprocessing, machine learning inference, and cloud synchronization.

Figure 2 illustrates the MQTT-based communication architecture used in the proposed system.

The MQTT-based communication layer enables efficient real-time transmission between the acquisition device and processing layer with minimal latency

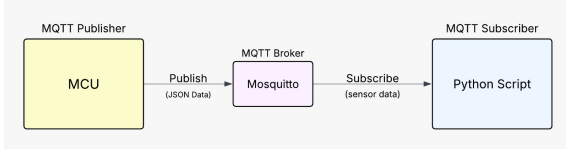


Figure 2: System Architecture

Table 3: Communication Configuration Parameters

Parameter	Value
Protocol	MQTT
QoS Level	1
Publish Interval	500 ms
Communication Medium	WiFi
Data Format	JSON
Topic Structure	factory/machine01/sensors

and network overhead.

3.4 Edge Computing Node

The edge computing node is implemented locally on a laptop system using Python and Visual Studio Code. The edge node acts as the primary intelligence layer of the proposed predictive maintenance framework (Artiushenko et al., 2024), (Savaglio et al., 2024). It performs data preprocessing, dataset generation, machine learning inference, anomaly detection, Remaining Useful Life (RUL) prediction, cloud synchronization, and dashboard integration.

The edge computing architecture consists of four major modules including the MQTT subscriber module, preprocessing module, machine learning inference module, and Firebase cloud integration module.

3.4.1 MQTT Subscriber Module

The MQTT subscriber module continuously receives live sensor data transmitted from the ESP32-based DAQ system. The subscriber application parses incoming JSON packets and performs real-time logging of sensor readings into CSV files. These CSV datasets are later utilized for machine learning model training, historical analysis, and predictive maintenance research.

3.4.2 Dataset Collection and Preparation

Real-time sensor data captured from the IIoT setup with and without faults is used as input to the proposed model. Around 10000 samples have been obtained from sensors such as temperature, vibration, RPM, voltage, and current with the help of a data acquisition system based on STM32 technology. The

preprocessing of the data includes steps like normalization, noise reduction, and feature extraction using the sliding window approach before being fed to the machine learning algorithm. An 80-20 split was performed for training and testing with 5-fold cross-validation.

3.4.3 Data Preprocessing Module

The preprocessing module processes incoming raw sensor readings before machine learning analysis. The preprocessing pipeline includes noise filtering, missing value handling, normalization, signal conditioning, feature extraction, and data formatting.

Sliding window techniques are used to convert continuous sensor signals into feature vectors suitable for machine learning inference. Statistical and time-domain features are extracted from the sensor data to identify degradation patterns and abnormal machine behavior.

The extracted statistical features include mean, standard deviation, and Root Mean Square (RMS).

Mean value of sensor readings is calculated using:

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i \quad (1)$$

where μ represents the average sensor value, x_i represents the individual sensor sample, and N represents the total number of samples.

Standard deviation is calculated as:

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2} \quad (2)$$

where σ represents the variation in sensor readings.

Root Mean Square (RMS) value is calculated as:

$$RMS = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2} \quad (3)$$

where RMS represents the effective magnitude of the sensor signal.

These extracted features improve the capability of the machine learning models to identify machine degradation and abnormal operating conditions.

3.5 Machine Learning Inference Layer

The processed feature vectors are passed to multiple machine learning models for fault classification, anomaly detection, and predictive analysis. The proposed framework utilizes a hybrid machine learning architecture consisting of Random Forest, XGBoost, Isolation Forest, and Long Short-Term Mem-

ory (LSTM) models(Taşcı et al., 2023),(Elkateb et al., 2024),(Rakholia et al., 2025).

Random Forest and XGBoost algorithms are used for supervised fault classification due to their high classification accuracy, robustness against noisy sensor data, and fast inference capability. These models classify machine conditions into normal and faulty operating states based on extracted sensor features.

Isolation Forest is implemented for unsupervised anomaly detection(Cummins et al., 2024). The model identifies abnormal operational patterns and detects outlier conditions that deviate from normal machine behavior. This enables early-stage anomaly detection even in cases where labeled fault data is limited.

The LSTM neural network model is used for time-series forecasting and Remaining Useful Life (RUL) estimation(Taşcı et al., 2023). Since industrial sensor data exhibits temporal dependencies, the LSTM model effectively captures sequential machine behavior and predicts future degradation trends.

The machine learning models were trained using historical sensor datasets collected under both normal and faulty machine conditions. The collected dataset included vibration, temperature, voltage, current, and RPM measurements generated during continuous machine operation. The dataset was divided into training and testing sets using an 80:20 split ratio.

Table 4 summarizes the machine learning model parameters used in the proposed system.

Table 4: Machine Learning Model Parameters

Parameter	Value
Fault Classification Models	Random Forest, XGBoost
Anomaly Detection Model	Isolation Forest
Time-Series Prediction Model	LSTM
Number of Features	18
Training Dataset Size	10000 Samples
Training Split	80%
Testing Split	20%
Validation Method	5-Fold Cross Validation

The machine learning inference module generates outputs including fault classification results, anomaly scores, predicted machine conditions, future degradation trends, and Remaining Useful Life estimations.

3.6 Cloud Database and Data Synchronization

Firebase Realtime Database is utilized as the cloud storage and synchronization platform within the proposed architecture. Firebase stores real-time sensor readings, processed datasets, machine learning out-

puts, anomaly scores, and predicted machine health information.

The edge computing node continuously uploads both live sensor data and machine learning prediction results to Firebase using REST API requests. The cloud database acts as a centralized storage platform enabling remote monitoring, synchronized data access, and future dataset retrieval for model retraining purposes.

The cloud storage structure is logically divided into the following categories:

- Real-Time Sensor Data Storage
- Processed Data Storage
- ML Prediction Storage
- Historical Fault Logs
- Remaining Useful Life Records

This layered storage structure improves data organization, scalability, and retrieval efficiency for industrial analytics applications.

3.7 Dashboard and Visualization Layer

The dashboard layer provides real-time visualization and remote monitoring of industrial machine conditions. The web dashboard application is developed locally using Python-based technologies and continuously retrieves live sensor readings and machine learning predictions from Firebase Realtime Database through REST API communication.

The dashboard supports dynamic real-time updates without requiring webpage refresh. Various visualization components including live charts, gauges, health indicators, status panels, and trend graphs are integrated to improve operator understanding of machine conditions.

The dashboard provides the following functionalities:

- Real-time sensor monitoring
- Historical trend visualization
- Fault prediction display
- Machine health estimation
- Remaining Useful Life visualization
- Alert and notification generation
- Anomaly status indication

Whenever abnormal machine conditions are detected, the system automatically generates alerts and notifications indicating the machine ID, fault severity, predicted failure type, and maintenance recommendation. This enables maintenance teams to take

corrective actions before catastrophic machine failure occurs.

The proposed architecture provides a scalable, low-cost, and real-time Industrial IoT predictive maintenance solution suitable for practical industrial deployment and smart manufacturing environments.

4 RESULT & DISCUSSION

4.1 Real-Time Sensor Monitoring Analysis

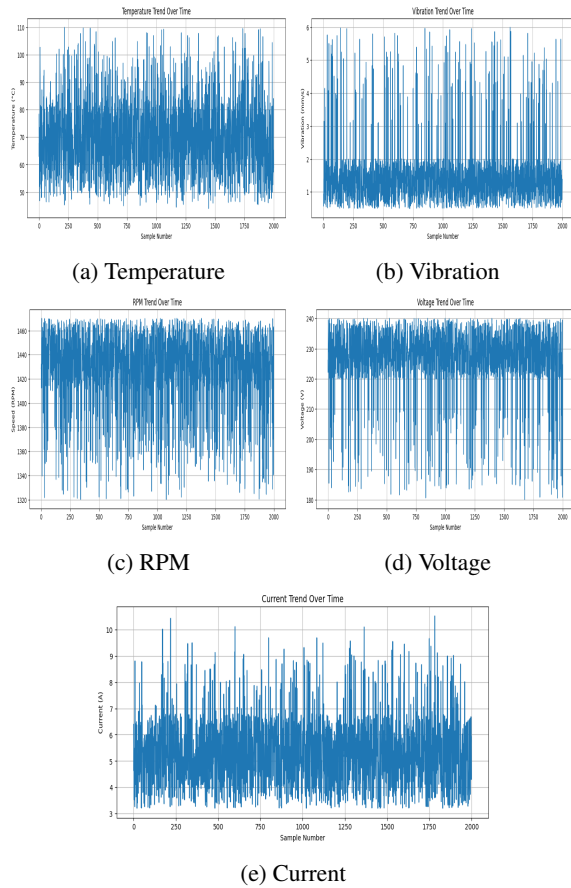


Figure 3: Real-time sensor monitoring results obtained from (a) Temperature, (b) Vibration, (c) RPM, (d) Voltage, and (e) Current sensors

The proposed predictive maintenance system was evaluated using real-time sensor data collected from temperature, vibration, RPM, voltage, and current sensors. Approximately 2000 samples were analyzed to monitor machine behavior and identify operational variations.

Temperature measurements varied between 45°C

and 110°C, where occasional spikes indicated possible heating effects caused by changing load conditions. Vibration values were generally maintained within 0.5 to 2 mm/s, while sudden peaks up to 6 mm/s suggested abnormal operating conditions such as bearing wear or shaft misalignment.

RPM readings ranged from 1320 to 1470 RPM, demonstrating stable machine operation with minor fluctuations during varying load conditions. Voltage levels were mostly maintained between 220 to 240 V, with small variations indicating minor power supply disturbances. Similarly, current consumption ranged from 3 to 10.5 A, where increased current values represented higher motor loading conditions.

Overall, the multi-sensor monitoring framework successfully captured real-time machine behavior and demonstrated effective continuous condition monitoring capability for predictive maintenance applications.

4.2 Machine Learning Model Performance Analysis

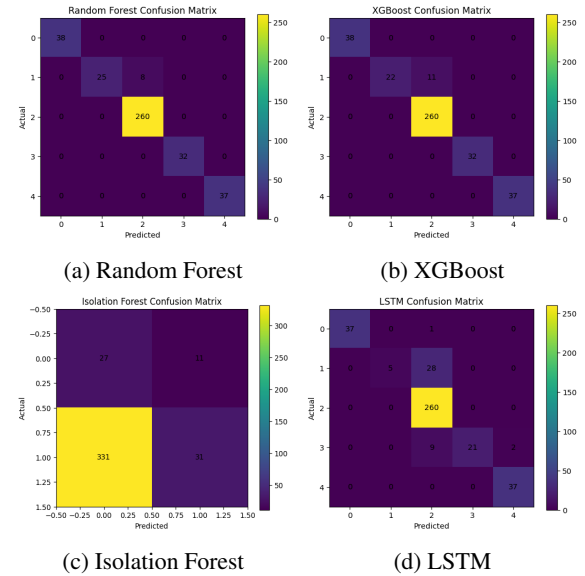


Figure 4: Confusion matrices of (a) Random Forest, (b) XGBoost, (c) Isolation Forest, and (d) LSTM models

The confusion matrices presented in Figure 4 demonstrate the classification performance of the implemented machine learning models for industrial fault detection. The Random Forest and XGBoost models achieved high classification accuracy with minimal misclassification errors, indicating effective fault prediction capability. The Isolation Forest model successfully identified abnormal operating conditions with fewer false detections, making it

suitable for anomaly detection tasks. Similarly, the LSTM model effectively captured temporal variations in machine behavior and provided reliable prediction performance.

The higher concentration of values along the diagonal regions of the confusion matrices indicates accurate classification results and validates the effectiveness of the proposed predictive maintenance framework for real-time industrial monitoring.

Table 5 presents the comparative performance analysis of the implemented machine learning models using accuracy, precision, recall, F1-score, and inference latency metrics.

Table 5: Performance Comparison of ML Models

Model	Accuracy	Precision	Recall	F1-Score	Latency
RF	94.2	93.8	94.5	94.1	320
XGB	93.4	92.9	93.7	93.3	280
IF	88.1	87.4	86.9	87.1	190
LSTM	93.1	92.5	92.8	92.6	1180

Overall, the Random Forest model achieved the best balance between classification accuracy and computational efficiency, while XGBoost provided reliable prediction performance with lower inference latency. Although the LSTM model demonstrated effective temporal learning capability, it required higher processing time due to sequential data analysis.

4.3 Real-Time Dashboard and System Visualization

The proposed Nirikshak framework includes a web-based real-time monitoring dashboard for continuous visualization of industrial machine parameters. Figures 5 and 6 present the developed interface integrated with machine learning-based fault prediction and cloud synchronization.

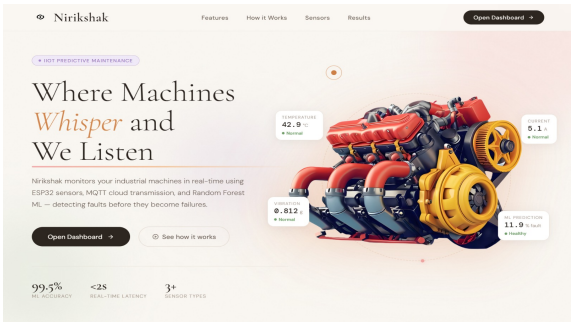


Figure 5: Web-based interface of the proposed Nirikshak framework

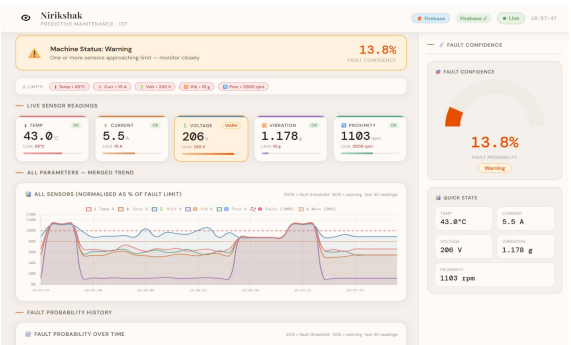


Figure 6: Real-time monitoring dashboard with fault analysis

The dashboard continuously displays temperature, vibration, voltage, current, and RPM sensor readings along with real-time fault probability estimation. In addition, merged trend visualization and warning indicators enable operators to identify abnormal machine behavior at an early stage. The integration of MQTT communication and Firebase cloud synchronization provides continuous remote monitoring capability with low-latency data updates.

The developed visualization framework improves operator awareness, enables proactive maintenance decision-making, and enhances the practical usability of the proposed predictive maintenance system in industrial environments.

4.4 Cost Analysis and Industrial Feasibility

Table 6: Estimated Hardware Cost Analysis

Component	Cost (INR)
STM32 Microcontroller	Rs.700
ESP8266 WiFi Module	Rs.400
Temperature Sensor	Rs.300
Vibration Sensor	Rs.1,000
RPM Sensor	Rs.500
Voltage and Current Sensors	Rs.900
PCB and Connectors	Rs.1,500
Power Supply	Rs.1,200
Miscellaneous Components	Rs.2,000
Total Hardware Cost	Rs.8,500

The proposed Nirikshak framework was developed using low-cost embedded hardware and open-source technologies. The total estimated implementation cost was approximately **Rs.8,500**, which is significantly lower than conventional predictive maintenance systems. The reduced implementation cost, low deployment complexity, and real-time monitoring ca-

pability make the proposed system economically feasible for small and medium-scale industries (SMEs). Furthermore, the modular architecture enables easy scalability for future industrial expansion and additional sensor integration.

5 CONCLUSIONS

The proposed *Nirikshak* framework presents an Industrial IoT-based predictive maintenance system designed for real-time machine monitoring and early fault detection. The framework integrates multi-sensor data acquisition, STM32-based edge processing, MQTT communication, machine learning inference, Firebase cloud synchronization, and real-time dashboard visualization into a unified architecture.

During the experiments conducted, the method proved highly efficient with a 94.2% rate of accurate fault classifications, with the latency for data transfer measured to be about 320 milliseconds. Moreover, the models were capable of detecting anomalies and deterioration in a timely manner before a catastrophic failure occurred. Furthermore, the system's construction cost only Rs. 8,500, which is a rather low price compared to that for other commercially available platforms.

As such, it can be argued that the presented system is a reliable proof-of-concept demonstrating how advanced predictive maintenance technology can be implemented using low-cost embedded hardware and open-source software. It is scalable, modular, and versatile enough to be implemented across many kinds of industries.

Future research directions include working on lightweight edge AI models, adaptive thresholding for better efficiency, online machine learning model retraining, improvements in cybersecurity measures, and implementation in large-scale industrial environments comprising multiple machines.

REFERENCES

- Aissani, S., Omar, M., and Hamouid, K. (2024). A comprehensive review of predictive maintenance technologies for vehicle reliability. In *International Conference on Computational Science and Computational Intelligence*, pages 56–67. Springer.
- Andriulo, F. C., Fiore, M., Mongiello, M., Traversa, E., and Zizzo, V. (2024). Edge computing and cloud computing for internet of things: A review. In *Informatics*, volume 11, page 71. MDPI.
- Artiushenko, V., Lang, S., Lerez, C., Reggelin, T., and Hackert-Oschätzchen, M. (2024). Resource-efficient edge ai solution for predictive maintenance. *Procedia Computer Science*, 232:348–357.
- Benhanifia, A., Cheikh, Z. B., Oliveira, P. M., Valente, A., and Lima, J. (2025). Systematic review of predictive maintenance practices in the manufacturing sector. *Intelligent Systems with Applications*, 26:200501.
- Cummins, L., Sommers, A., Ramezani, S. B., Mittal, S., Jabour, J., Seale, M., and Rahimi, S. (2024). Explainable predictive maintenance: A survey of current methods, challenges and opportunities. *IEEE access*, 12:57574–57602.
- Elkateb, S., Métwalli, A., Shendy, A., and Abu-Elanien, A. E. (2024). Machine learning and iot-based predictive maintenance approach for industrial applications. *Alexandria Engineering Journal*, 88:298–309.
- Hamasha, M., Albedoor, Q., Hamasha, S., Ali, H., Qamar, A., and Barrah, F. (2025). A comprehensive framework for iot-driven predictive maintenance: Leveraging ai and edge computing for enhanced equipment reliability. *Journal of Applied Engineering Science*, 23(3):471–486.
- Rakholia, R., Suárez-Cetrulo, A. L., Singh, M., and Carbajo, R. S. (2025). Integrating ai and iot for predictive maintenance in industry 4.0 manufacturing environments: A practical approach. *Information*, 16(9):737.
- Samatas, G. G., Moumgiakmas, S. S., and Papakostas, G. A. (2021). Predictive maintenance-bridging artificial intelligence and iot. In *2021 IEEE World AI IoT Congress (AIIoT)*, pages 0413–0419. IEEE.
- Savaglio, C., Mazzei, P., and Fortino, G. (2024). Edge intelligence for industrial iot: Opportunities and limitations. *Procedia Computer Science*, 232:397–405.
- Taşcı, B., Omar, A., and Ayvaz, S. (2023). Remaining useful lifetime prediction for predictive maintenance in manufacturing. *Computers & Industrial Engineering*, 184:109566.
- Theissler, A., Pérez-Velázquez, J., Kettelgerdes, M., and Elger, G. (2021). Predictive maintenance enabled by machine learning: Use cases and challenges in the automotive industry. *Reliability engineering & system safety*, 215:107864.